

Машинное обучение в финансах

Лекция 3: Введение в RL

Игорь Удовиченко

Факультатив Веги

10 апреля 2026 г.

Источник

Лекция содрана с великолепнейшего конспекта по RL:



<https://arxiv.org/abs/2201.09746>

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
5. Параметрическая Q-функция
6. Policy Gradient
7. RL для хеджирования деривативов

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
5. Параметрическая Q-функция
6. Policy Gradient
7. RL для хеджирования деривативов

Марковская цепь и среда

Марковская цепь — пара $(\mathcal{S}, \mathcal{P})$:

- $\mathcal{S}$ — множество состояний
- $\mathcal{P}$ — вероятности переходов $p(s_{t+1} \mid s_t)$

Цепь **стационарна**, если $\forall t: p(s_{t+1} \mid s_t) = p(s' \mid s)$ не зависит от t .

Среда (environment) — тройка $(\mathcal{S}, \mathcal{A}, \mathcal{P})$:

- $\mathcal{S}$ — *пространство состояний* (state space)
- $\mathcal{A}$ — *пространство действий* (action space)
- $\mathcal{P}$ — *функция переходов*: $p(s' \mid s, a)$

Среда **марковская** (зависит только от текущего s , не от истории) и **стационарная** (не зависит от t).

Траектория и MDP

Стратегия (policy): $\pi(a \mid s)$ — распределение действий в состоянии s .

Траектория: $\mathcal{T} := (s_0, a_0, s_1, a_1, s_2, a_2, \dots)$

Распределение на траекториях:

$$p(\mathcal{T}) = \prod_{t \geq 0} \pi(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t)$$

Марковский процесс принятия решений (MDP) — четвёрка $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r)$:

- $\mathcal{S}, \mathcal{A}, \mathcal{P}$ — среда
- $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ — *функция награды* (reward function)

Процесс: $s_0 \xrightarrow{a_0 \sim \pi} r_0 = r(s_0, a_0), s_1 \sim p(\cdot \mid s_0, a_0) \xrightarrow{a_1 \sim \pi} r_1, s_2 \sim p(\cdot \mid s_1, a_1) \rightarrow \dots$

Награда и целевой функционал

Суммарная награда траектории: $\sum_{t \geq 0} r_t$, где $r_t := r(s_t, a_t)$

Проблема: сумма может расходиться $\Rightarrow$ вводим **дисконтирование**, $\gamma \in (0, 1)$:

$$R(\mathcal{T}) := \sum_{t \geq 0} \gamma^t r_t$$

Зачем нужно $\gamma < 1$:

- гарантирует ограниченность суммы: $|R(\mathcal{T})| \leq \frac{r^{\max}}{1-\gamma}$
- приоритет ближайшей награды перед отложенной
- интерпретация: на каждом шаге с вероятностью $1 - \gamma$ процесс обрывается

Целевой функционал (score) задачи RL:

$$J(\pi) := \mathbb{E}_{\mathcal{T} \sim \pi} R(\mathcal{T}) = \mathbb{E}_{\mathcal{T} \sim \pi} \sum_{t \geq 0} \gamma^t r_t \longrightarrow \max_{\pi}$$

План лекции

1. Модель взаимодействия агента со средой
- 2. Алгоритмы обучения с подкреплением**
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
5. Параметрическая Q-функция
6. Policy Gradient
7. RL для хеджирования деривативов

Пространства состояний и действий

Пространство состояний $\mathcal{S}$:

- *дискретное* (табличное): $|\mathcal{S}| < +\infty$ — например, конфигурации на доске
- *непрерывное*: $\mathcal{S} \subseteq \mathbb{R}^d$ — например, координаты и скорости робота

Пространство действий $\mathcal{A}$:

- *дискретное*: $|\mathcal{A}| < +\infty$ — например, кнопки джойстика
- *непрерывное*: $\mathcal{A} \subseteq \mathbb{R}^m$ — задачи *непрерывного управления* (continuous control), например, моменты в сочленениях робота

Вид пространств определяет, какие алгоритмы применимы.

On-policy vs Off-policy

Behavior policy — стратегия сбора данных во время обучения.

Target policy — итоговая стратегия, которую мы хотим выучить.

Off-policy алгоритм может обучаться на опыте *произвольной* стратегии.

- Можно переиспользовать старые данные, данные эксперта и т.д.
- Учится на чужих ошибках

On-policy алгоритму для очередной итерации нужен опыт *конкретной*, предоставляемой самим алгоритмом, стратегии.

- Не может переиспользовать данные с прошлых итераций
- Учится на своих ошибках

Классификация RL-алгоритмов

Класс	Идея	Особенности
Мета-эвристики	Black-box оптимизация $J(\pi) \rightarrow \max$	Любые $\mathcal{S}$, $\mathcal{A}$; низкая sample efficiency
Value-based	Оптимальная стратегия через оценочные функции	Дискретные $\mathcal{A}$; off-policy
Policy gradient	Градиент $J(\pi)$ по параметрам π	Любые $\mathcal{A}$; on-policy
Model-based	Учим / используем модель $p(s' s, a)$	Любые $\mathcal{S}$, $\mathcal{A}$; максимальная sample efficiency

Sample efficiency (от низкой к высокой):

Мета-эвристики $\rightarrow$ Policy Gradient $\rightarrow$ Value-based $\rightarrow$ Model-based

Разреженная награда и Reward Shaping

Проблема: хорошая награда поощряет за результат, а не за способ $\Rightarrow$ обычно она *разреженная* (sparse), и агент редко получает ненулевой сигнал.

Reward Shaping: замена награды с помощью потенциала $\Phi: \mathcal{S} \rightarrow \mathbb{R}$, $\Phi(s_T) = 0$:

$$r^{\text{new}}(s, a, s') := r(s, a) + \gamma\Phi(s') - \Phi(s)$$

Теорема. Reward shaping не меняет задачу.

Доказательство. Добавка к суммарной награде траектории:

$$\sum_{t \geq 0} [\gamma^{t+1}\Phi(s_{t+1}) - \gamma^t\Phi(s_t)] = -\Phi(s_0)$$

Это телескопирующаяся сумма: все промежуточные члены сокращаются; $\lim_{t \rightarrow \infty} \gamma^t\Phi(s_t) = 0$ в силу $\gamma < 1$ и ограниченности Φ . Слагаемое $-\Phi(s_0)$ не зависит от π и не влияет на $\arg \max_{\pi} J(\pi)$. ■

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
- 3. Оценочные функции и уравнения Беллмана**
4. Табличные алгоритмы
5. Параметрическая Q-функция
6. Policy Gradient
7. RL для хеджирования деривативов

V-функция и Q-функция

Reward-to-go с момента t : $R_t := \sum_{\hat{t} \geq t} \gamma^{\hat{t}-t} r_{\hat{t}}$

V-функция (state value function) стратегии π :

$$V^\pi(s) := \mathbb{E}_{\mathcal{T} \sim \pi | s_0=s} R(\mathcal{T})$$

Сколько *в среднем* наберёт агент, стартуя из s и действуя по π .

Q-функция (state-action value function) стратегии π :

$$Q^\pi(s, a) := \mathbb{E}_{\mathcal{T} \sim \pi | s_0=s, a_0=a} R(\mathcal{T})$$

Сколько наберёт агент, если из s сначала выполнит a , а дальше действует по π .

Связь V и Q:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(a|s)} Q^\pi(s, a) \tag{VQ}$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} V^\pi(s') \tag{QV}$$

Уравнения Беллмана

Подставляя связи друг в друга, получаем **уравнения Беллмана**:

VV-уравнение:

$$V^\pi(s) = \mathbb{E}_a[r(s, a) + \gamma \mathbb{E}_{s'} V^\pi(s')] \quad (\text{VV})$$

QQ-уравнение:

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \mathbb{E}_{a'} Q^\pi(s', a') \quad (\text{QQ})$$

Интуиция: награда за игру = награда за шаг + дисконтированная награда за остаток.

V^π — единственное решение VV-уравнения

Q^π — единственное решение QQ-уравнения.

Оптимальные оценочные функции

Оптимальные V- и Q-функции:

$$V^*(s) := \max_{\pi} V^{\pi}(s) \quad Q^*(s, a) := \max_{\pi} Q^{\pi}(s, a)$$

Связь оптимальных функций (принцип оптимальности Беллмана):

$$V^*(s) = \max_a Q^*(s, a) \quad (V^*Q^*)$$

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} V^*(s') \quad (Q^*V^*)$$

Уравнения оптимальности Беллмана:

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \max_{a'} Q^*(s', a') \quad (Q^*Q^*)$$

$$V^*(s) = \max_a [r(s, a) + \gamma \mathbb{E}_{s'} V^*(s')] \quad (V^*V^*)$$

Критерий оптимальности Беллмана

Теорема. Стратегия π оптимальна тогда и только тогда, когда $\forall s, a: \pi(a | s) > 0$:

$$a \in \operatorname{Argmax}_a Q^\pi(s, a)$$

Иначе: π оптимальна $\Leftrightarrow \pi$ жадна по своей же Q-функции.

Ключевые следствия:

- Если $|\mathcal{A}| < +\infty$, оптимальная стратегия всегда существует
- Если существует оптимальная стратегия, то существует *детерминированная* оптимальная стратегия: $\pi^*(s) = \arg \max_a Q^*(s, a)$
- Q^* — единственное решение уравнения оптимальности Беллмана $\Rightarrow$ вместо поиска π^* можно искать Q^*

Почему это работает: оптимальность можно доказать через отказ от однородности — рассмотреть нестационарные стратегии $\pi_t(a | s)$, для которых MDP превращается в дерево и жадный выбор на каждом шаге очевидно оптимален. Затем показать, что оптимум по нестационарным стратегиям совпадает со стационарным.

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
- 4. Табличные алгоритмы**
5. Параметрическая Q-функция
6. Policy Gradient
7. RL для хеджирования деривативов

Стохастическая аппроксимация

В model-free режиме $p(s' | s, a)$ неизвестна — мы не можем взять $\mathbb{E}_{s'}$.

Но уравнения оптимальности Беллмана похожи на **стохастическую аппроксимацию**:

$$x = \mathbb{E}_\varepsilon f(x, \varepsilon)$$

где ε — сэмпл из неизвестного распределения.

Решение: экспоненциальное сглаживание с learning rate α_k :

$$x_k = (1 - \alpha_k) x_{k-1} + \alpha_k f(x_{k-1}, \varepsilon), \quad \varepsilon \sim p(\varepsilon)$$

Эквивалентная форма («стохастический градиент»):

$$x_k = x_{k-1} + \alpha_k (f(x_{k-1}, \varepsilon) - x_{k-1})$$

Сходится при **условиях Роббинса–Монро**: $\sum_k \alpha_k = +\infty, \quad \sum_k \alpha_k^2 < +\infty$

Q-learning

Применяем стохастическую аппроксимацию к уравнению оптимальности Беллмана.

На каждом шаге собираем переход (s, a, r, s') и обновляем одну ячейку таблицы:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_k \left(\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{таргет}} - Q(s, a) \right)$$

Выражение $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ — **временная разность** (temporal difference).

Теорема (сходимость Q-learning).

Если $|S|, |A| < \infty$, каждая пара (s, a) встречается бесконечное количество раз (infinite visitation), а $\alpha_k(s, a)$ удовлетворяют условиям Роббинса–Монро, то $Q_k \rightarrow Q^*$ с вероятностью 1.

Off-policy: для обновления достаточно $s' \sim p(s' | s, a)$; стратегия сбора данных может быть любой (при условии infinite visitation). Например: ε -greedy.

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
- 5. Параметрическая Q-функция**
6. Policy Gradient
7. RL для хеджирования деривативов

От таблицы к нейросети

В сложных средах ($\mathcal{S} \subseteq \mathbb{R}^d$, или $|\mathcal{S}|$ велико) таблицу хранить нельзя. Приближаем $Q^*(s, a)$ нейросетью $Q_\theta(s, a)$.

Переход от стохастической аппроксимации к задаче регрессии:

- Вход: пара (s, a)
- Таргет: $y(s, a) := r + \gamma \max_{a'} Q_{\theta_k}(s', a')$, $s' \sim p(s' | s, a)$
- Функция потерь: $\frac{1}{2} (y - Q_\theta(s, a))^2 \rightarrow \min_\theta$

В среднем $\mathbb{E}_{s'} y(s, a) = r + \gamma \mathbb{E}_{s'} \max_{a'} Q_{\theta_k}(s', a')$ — правая часть уравнения оптимальности $\Rightarrow$ решение задачи регрессии есть шаг метода простой итерации.

Факт. Табличный Q-learning — частный случай при $Q_\theta(s, a) = \theta_{s,a}$.

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
5. Параметрическая Q-функция
- 6. Policy Gradient**
7. RL для хеджирования деривативов

Идея Policy Gradient

Value-based: ищем Q^* , стратегия неявно — $\pi(s) = \arg \max_a Q^*(s, a)$.

Policy Gradient: параметризуем стратегию $\pi_\theta(a | s)$ напрямую и оптимизируем:

$$J(\pi_\theta) = V^{\pi_\theta}(s_0) \rightarrow \max_{\theta}$$

Преимущества:

- Работает с непрерывными $\mathcal{A}$ без оптимизации $\max_a Q(s, a)$
- Может выучить стохастические стратегии
- Оптимизируем напрямую то, что нам нужно

Нужна формула градиента $\nabla_{\theta} J(\pi_{\theta})$.

Проблема: функция награды $r(s, a)$ не дифференцируема по a (например, $\mathcal{A}$ дискретно), а функция переходов $p(s' | s, a)$ неизвестна.

Решение: **log-derivative trick**.

Log-derivative trick и Policy Gradient Theorem

Для распределения $\pi_\theta(a)$, зависящего от θ :

$$\nabla_\theta \log \pi_\theta(a) = \frac{\nabla_\theta \pi_\theta(a)}{\pi_\theta(a)} \quad \Rightarrow \quad \nabla_\theta \mathbb{E}_{a \sim \pi_\theta} f(a) = \mathbb{E}_{a \sim \pi_\theta} \nabla_\theta \log \pi_\theta(a) f(a)$$

Градиент мат.ожидания стал мат.ожиданием $\Rightarrow$ можно оценивать по Монте-Карло.

Теорема (Policy Gradient Theorem).

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\mathcal{T} \sim \pi} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t | s_t) Q^\pi(s_t, a_t)$$

Интуиция: поощряем действия пропорционально их «качеству» $Q^\pi(s, a)$.

Ключевое свойство: $\nabla_\theta \log p(s' | s, a)$ в формуле нет — функция переходов не нужна.

REINFORCE

Заменяем $Q^\pi(s_t, a_t)$ на Монте-Карло оценку — reward-to-go R_t :

$$\nabla_\theta J(\pi_\theta) \approx \frac{1}{N} \sum_{\mathcal{T}} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t | s_t) R_t$$

Алгоритм REINFORCE: играем N эпизодов, считаем R_t в каждом, делаем шаг SGD.

Проблемы:

- On-policy: нужны свежие эпизоды на каждом шаге
- Нужно доигрывать до конца (эпизодичность)
- Огромная дисперсия: $Q^\pi(s, a)$ всюду положительна $\Rightarrow$ вероятность всех действий хочет увеличиться, а это невозможно.

Бэйзлайн

Ключевое свойство: $\mathbb{E}_{a \sim \pi_\theta} \nabla_\theta \log \pi_\theta(a) = 0$

$\Rightarrow$ можно вычесть любую функцию $b(s)$ от состояний:

$$\nabla_\theta J = \mathbb{E}_{\mathcal{T} \sim \pi} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t | s_t) (Q^\pi(s_t, a_t) - b(s_t))$$

Среднее не изменится, но дисперсия может сильно уменьшиться.

Advantage-функция: $A^\pi(s, a) := Q^\pi(s, a) - V^\pi(s)$

Свойство: $\mathbb{E}_{a \sim \pi} A^\pi(s, a) = 0$ — стимул центрирован.

$V^\pi(s)$ — хороший (близкий к оптимальному) бэйзлайн. Тогда:

$$\nabla_\theta J(\pi) = \mathbb{E}_{\mathcal{T} \sim \pi} \sum_{t \geq 0} \gamma^t \nabla_\theta \log \pi_\theta(a_t | s_t) A^\pi(s_t, a_t)$$

Хорошие действия ($A > 0$) — увеличиваем вероятность, плохие ($A < 0$) — уменьшаем.

Actor-Critic

Учим две модели:

- **Актёр** (actor): стратегия $\pi_{\theta}(a \mid s)$
- **Критик** (critic): $V_{\phi}(s) \approx V^{\pi}(s)$

V-критик используется и как бэйзлайн, и для оценки Q-функции:

$$Q^{\pi}(s, a) \approx r(s, a) + \gamma V_{\phi}(s'), \quad s' \sim p(s' \mid s, a)$$

Одношаговая оценка Advantage:

$$\Psi(s, a) = r + \gamma V_{\phi}(s') - V_{\phi}(s)$$

Актёр: градиентный подъём по θ с оценкой $\nabla_{\theta} \log \pi_{\theta}(a \mid s) \Psi(s, a)$

Критик: регрессия $V_{\phi}(s) \rightarrow y_V$, где $y_V = r + \gamma V_{\phi}(s')$ (или N -шаговая, или R_t)

План лекции

1. Модель взаимодействия агента со средой
2. Алгоритмы обучения с подкреплением
3. Оценочные функции и уравнения Беллмана
4. Табличные алгоритмы
5. Параметрическая Q-функция
6. Policy Gradient
- 7. RL для хеджирования деривативов**

Хеджирование как MDP

Продали дериватив с выплатой $c(S_T)$ в момент T . Хотим хеджировать позицию в базовом активе.

Состояние: $s_t = (S_t, t)$ — цена акции и текущий момент времени.

$\mathcal{S} \subseteq \mathbb{R}_+ \times \{0, 1, \dots, T\}$ — непрерывно-дискретное.

Действие: $h_t \in \mathbb{R}$ — количество акций, которые держим с t до $t + 1$.

$\mathcal{A} = \mathbb{R}$ — непрерывное (continuous control).

Функция переходов: $S_{t+1} \sim p(S_{t+1} \mid S_t)$ — модель динамики цены.

Нестационарность: переходы зависят от t (например, волатильность может меняться к экспирации). Марковская цепь *неоднородна*, но время t включено в состояние, поэтому расширенная цепь формально стационарна.

Эпизодичность: эпизод всегда длится ровно T шагов.

Награда и функционал

R&L позиции за один шаг: агент держит h_t акций с t до $t + 1$.

При $t < T - 1$ награда:

$$r_t = h_t(S_{t+1} - S_t)$$

В последний момент $t = T - 1$ дополнительно выплата по деривативу:

$$r_{T-1} = h_{T-1}(S_T - S_{T-1}) - c(S_T)$$

Стохастическая награда: r_t зависит от S_{t+1} — случайного следующего состояния. В стандартной формулировке MDP награда $r(s, a)$ детерминирована; здесь же $r_t = r(s_t, h_t, s_{t+1})$.

Это не проблема: стохастическую награду можно «вшить» в функцию переходов. Уравнения Беллмана сохраняются, если награду внести под мат.ожидание по s' :

$$Q^\pi(s, a) = \mathbb{E}_{s'} [r(s, a, s') + \gamma V^\pi(s')]$$

Дисконтирование возникает естественным образом.

Целевой функционал хеджирования

Суммарная награда — P&L хеджированного портфеля:

$$R = \sum_{t=0}^{T-1} h_t(S_{t+1} - S_t) - c(S_T)$$

Задача хеджирования: хотим заработать. В терминах RL:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{T-1} h_t(S_{t+1} - S_t) - c(S_T) \right] \rightarrow \max_{\pi}$$

Проблема! Если в модели цены есть *дрифт*, $\mathbb{E}[S_{t+1} - S_t \mid S_t] = \mu_t \neq 0$, то:

$$\mathbb{E}[h_t(S_{t+1} - S_t) \mid S_t] = h_t \cdot \mu_t$$

При отсутствии ограничений на h_t оптимальное действие $h_t^* = \pm\infty$ (знак определяется знаком μ_t). Функционал $J(\pi) = +\infty$ — задача вырождена. Агент “забывает” про хеджирование и тупо залетает на всю котлету.

Почему классическая постановка RL не работает

Максимизация $\mathbb{E}[R]$ — неправильная цель для хеджирования:

- Хеджер хочет *минимизировать риск*, а не максимизировать среднюю прибыль
- При наличии дрифта задача $\mathbb{E}[R] \rightarrow \max$ имеет тривиальное и бесполезное решение $|h| \rightarrow \infty$

Уравнения Беллмана формально работают, но бесполезны:

- $V^*(S_t, t) = +\infty$ для всех t и S_t — нет содержательного решения
- Q-learning сойдётся к $Q^* = +\infty$
- Policy gradient будет увеличивать $|h_t|$ без ограничений

Нужна *другая целевая функция*, учитывающая риск: не $\mathbb{E}[R] \rightarrow \max$, а, например, функция полезности.

Как это сделать — на следующей лекции.

Вопросы?